

Department of Electrical and Computer Engineering

ENCS4370 | Computer Architecture

Spring Semester 2025/2026

Project No. 2

Deadline: June 20, 2026

1. Objectives

- To design the Datapath and control path of a simple 32-bit RISC processor.
- To implement the processor using Verilog HDL.
- To verify the functionality of the processor through simulation.
- To design and test a custom instruction set architecture (ISA).
- To gain experience in processor microarchitecture design and verification.

2. Processor Specifications

1. The instruction size and word size are 32 bits.
 2. The processor contains sixteen 32-bit general-purpose registers, R0 through R15.
 3. R0 is hardwired to zero. Any attempt to write to R0 must be ignored.
 4. All negative values are represented using two's complement notation.
 5. The processor supports three instruction formats:
 - R-Type
 - I-Type
 - J-Type
-

3. Instruction Formats

R-Type (Register Type)

Opcode ⁶	Rd ⁴	Rs ⁴	Rt ⁴	Unused ¹⁴
---------------------	-----------------	-----------------	-----------------	----------------------

- 6-bit opcode
 - 4-bit destination register (Rd)
 - 4-bit source register (Rs)
 - 4-bit source register (Rt)
-

I-Type (Immediate Type)

Opcode ⁶	Rt ⁴	Rs ⁴	Immediate ¹⁸
---------------------	-----------------	-----------------	-------------------------

- 6-bit opcode
 - 4-bit destination/source register (Rt)
 - 4-bit source register (Rs)
 - 18-bit immediate value
 - Immediate values are sign-extended for arithmetic, memory, and branch instructions.
 - Immediate values are zero-extended for logical instructions.
-

J-Type (Jump Type)

Opcode ⁶	Offset ²⁶
---------------------	----------------------

- 6-bit opcode
 - 26-bit signed offset
 - The target address is computed by adding the sign-extended offset to the current PC.
-

4. Instruction Encoding

The following instructions must be implemented.

R-Type Instructions

Instruction	Meaning	Opcode
ADD Rd, Rs, Rt	$\text{Reg(Rd)} = \text{Reg(Rs)} + \text{Reg(Rt)}$	0
SUB Rd, Rs, Rt	$\text{Reg(Rd)} = \text{Reg(Rs)} - \text{Reg(Rt)}$	1
AND Rd, Rs, Rt	$\text{Reg(Rd)} = \text{Reg(Rs)} \& \text{Reg(Rt)}$	2
OR Rd, Rs, Rt	$\text{Reg(Rd)} = \text{Reg(Rs)} \mid \text{Reg(Rt)}$	3
XOR Rd, Rs, Rt	$\text{Reg(Rd)} = \text{Reg(Rs)} \wedge \text{Reg(Rt)}$	4
NOR Rd, Rs, Rt	$\text{Reg(Rd)} = \sim(\text{Reg(Rs)} \mid \text{Reg(Rt)})$	5
SLL Rd, Rs, Rt	$\text{Reg(Rd)} = \text{Reg(Rs)} \ll \text{Reg(Rt)}$	6
SRL Rd, Rs, Rt	$\text{Reg(Rd)} = \text{Reg(Rs)} \gg \text{Reg(Rt)}$	7
CLR Rd	$\text{Reg(Rd)} = 0$	8
JR Rs	$\text{PC} = \text{Reg(Rs)}$	9

I-Type Instructions

Instruction	Meaning	Opcode
ADDI Rt, Rs, Imm	$\text{Reg(Rt)} = \text{Reg(Rs)} + \text{Imm}$	10
ANDI Rt, Rs, Imm	$\text{Reg(Rt)} = \text{Reg(Rs)} \& \text{Imm}$	11
ORI Rt, Rs, Imm	$\text{Reg(Rt)} = \text{Reg(Rs)} \mid \text{Imm}$	12
LW Rt, Imm(Rs)	$\text{Reg(Rt)} = \text{Mem}(\text{Reg(Rs)} + \text{Imm})$	13
SW Rt, Imm(Rs)	$\text{Mem}(\text{Reg(Rs)} + \text{Imm}) = \text{Reg(Rt)}$	14
SWAP Rt, Imm(Rs)	Exchange Reg(Rt) with $\text{Mem}(\text{Reg(Rs)} + \text{Imm})$	15
BEQ Rt, Rs, Imm	Branch if $\text{Reg(Rt)} = \text{Reg(Rs)}$	16
BNE Rt, Rs, Imm	Branch if $\text{Reg(Rt)} \neq \text{Reg(Rs)}$	17

For branch instructions:

If condition is true:

$$PC = PC + \text{sign_extend}(\text{Imm})$$

Otherwise:

$$PC = PC + 1$$

J-Type Instructions

Instruction	Meaning	Opcode
J Label	Unconditional jump	18
JAL Label	Jump and link	19

For JAL:

$$R14 = PC + 1$$

$$PC = \text{Target Address}$$

5. RTL Design

Design and implement the processor using Verilog HDL.

A complete solution shall support all instructions listed above.

Students may implement the processor as:

- Multi-cycle processor (complete solution)
- 5-stage pipelined processor (bonus)

The pipelined implementation should consist of the following stages:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write Back (WB)

The design must include:

- Datapath
 - Control path
 - Register file
 - ALU
 - memory
-

6. Design Verification

To verify the RTL design:

1. Develop a complete Verilog testbench.
2. Create multiple assembly programs that exercise all instructions.
3. Convert the programs into machine code.
4. Load the programs into instruction memory.
5. Demonstrate correct processor execution through simulation.

The submitted tests must collectively verify all implemented instructions.

7. Project Report

The report must contain the following sections.

1 – Design and Implementation

1. Detailed description of the processor datapath.
2. Description of all processor components.
3. Description of the control signals.
4. Control truth tables.
5. Boolean equations and Logic diagrams.
6. Datapath and control-path diagrams.
7. Design decisions and justification.
8. References for any external sources used.

2 – Simulation and Testing

1. Description of all test programs.
 2. Expected output of each test.
 3. Simulation results.
 4. Waveform screenshots.
 5. List of instructions verified successfully.
 6. Discussion of any limitations or issues.
-

8. Teamwork

1. Work in groups of up to three students.
 2. Team members must participate in:
 - Design and implementation
 - Simulation and testing
 - Report writing
 - Project discussion
 3. Clearly identify the contribution of each team member.
-

9. Submission Guidelines

Submit a single ZIP file containing:

- Verilog source files
 - Testbench files
 - Assembly programs
 - Binary instruction files
 - Any supporting data files
 - Final report document
-